POMPS Presentation: 1.1.55

Subhodeep Chakraborty
ee25btech11055,
IIT Hyderabad.

March 25, 2026

1. Problem
2. Solution
3. Python Codes
4. Plots
   - Gaussian Pulse
   - Sinc Pulse

Problem Statement

Let the signal $x(t)$ have the Fourier transform $X(\omega)$. Consider the signal $y(t) = x(t - t_a)$ where t_a is an arbitrary delay. The magnitude of the Fourier transform of $y(t)$ is:

a. $|X(\omega) \cdot [\omega]|$
b. $|X(\omega)| \cdot \omega$
c. $\omega^2 \cdot |X(\omega)|$
d. $\left|X(\omega) \cdot e^{-j\omega t_a}\right|$

Theoretical Solution

Let $x(t)$ be mapped to the frequency domain via the **Continuous Fourier Transform Operator** $\mathcal{F}$, such that $x(t) \overset{\mathcal{F}}{\rightleftharpoons} X(\omega)$.

According to the **Time-Shifting Theorem**, a delay in the time domain corresponds to a phase rotation in the frequency domain, leaving the magnitude unchanged.

Mathematically:

$$x(t - t_a) \overset{\mathcal{F}}{\rightleftharpoons} X(\omega)e^{-j\omega t_a} \tag{3.1}$$

Proof: Time-Shifting Theorem

By the definition of the Continuous-Time Fourier Transform:

$$\mathcal{F}\left\{x(t-t_a)\right\} = \int_{-\infty}^{\infty} x(t-t_a)e^{-j\omega t}dt \tag{3.2}$$

Let $\tau = t - t_a$.

$$\mathcal{F}\left\{x(t-t_a)\right\} = \int_{-\infty}^{\infty} x(\tau)e^{-j\omega(\tau+t_a)}d\tau \tag{3.3}$$

Proof: Time-Shifting Theorem

$$\mathcal{F}\left\{x(t-t_a)\right\} = \int_{-\infty}^{\infty} x(\tau)e^{-j\omega\tau}e^{-j\omega t_a}d\tau \tag{3.4}$$

$$\mathcal{F}\left\{x(t-t_a)\right\} = e^{-j\omega t_a}\int_{-\infty}^{\infty} x(\tau)e^{-j\omega\tau}d\tau \tag{3.5}$$

$$\mathcal{F}\left\{x(t-t_a)\right\} = e^{-j\omega t_a}X(\omega) \tag{3.6}$$

Eigenfunctions in LTI Systems

To map this to linear algebra, we recall that an eigenvector $\mathbf{v}$ does not change direction under a linear transformation T; it is only scaled by an eigenvalue λ, such that $T(\mathbf{v}) = \lambda\mathbf{v}$.

Similarly, if passing a signal $x(t)$ through a Linear Time-Invariant (LTI) system H simply scales the signal by a complex constant λ:

$$\mathcal{H}\{x(t)\} = \lambda \cdot x(t) \tag{3.7}$$

Then $x(t)$ is considered an **eigenfunction** of that system.

Proof: Complex Exponentials as Eigenfunctions

The output $y(t)$ of an LTI system is the convolution of the input signal $x(t)$ with the system's impulse response $h(t)$:

$$y(t) = h(t) * x(t) = \int_{-\infty}^{\infty} h(\tau)x(t-\tau)d\tau \tag{3.8}$$

Let our input signal be a general complex exponential $x(t) = e^{st}$. Substituting this into the integral:

$$y(t) = \int_{-\infty}^{\infty} h(\tau)e^{s(t-\tau)}d\tau \tag{3.9}$$

$$y(t) = \int_{-\infty}^{\infty} h(\tau)e^{st}e^{-s\tau}d\tau \tag{3.10}$$

Proof: Complex Exponentials (Cont.)

Because we are integrating with respect to τ, the term e^{st} is treated as a constant and factored out:

$$y(t) = e^{st} \int_{-\infty}^{\infty} h(\tau) e^{-s\tau} d\tau \tag{3.11}$$

The remaining integral evaluates to a complex number dependent only on s. This is the definition of the system's **Transfer Function**, $H(s)$:

$$H(s) = \int_{-\infty}^{\infty} h(\tau) e^{-s\tau} d\tau \tag{3.12}$$

Therefore, replacing the integral gives:

$$y(t) = H(s) \cdot e^{st} = H(s) \cdot x(t) \tag{3.13}$$

Linear Algebraic Formulation

Since complex exponentials are eigenfunctions of LTI systems, we can formalise the time-delay operation using linear algebra. Let the representation of the continuous frequency spectrum at discrete frequencies ω_k be a column vector $\mathbf{X}$.

The time-shift operation acts as a **Linear Transformation** mapping input spectrum $\mathbf{X}$ to output spectrum $\mathbf{Y}$. This transformation is represented by a diagonal Matrix $\mathbf{C}$:

$$\mathbf{Y} = \mathbf{C}\mathbf{X} \tag{3.14}$$

Matrix Expansion

Thus, from (3.1) we can write:

$$\begin{pmatrix} Y(\omega_1) \\ Y(\omega_2) \\ \vdots \\ Y(\omega_k) \end{pmatrix} = \begin{pmatrix} e^{-j\omega_1 t_a} & 0 & \dots & 0 \\ 0 & e^{-j\omega_2 t_a} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & e^{-j\omega_k t_a} \end{pmatrix} \begin{pmatrix} X(\omega_1) \\ X(\omega_2) \\ \vdots \\ X(\omega_k) \end{pmatrix} \tag{3.15}$$

Magnitude Evaluation

For any arbitrary frequency component ω, the operation simplifies to:

$$|Y(\omega)| = \left|X(\omega) \cdot e^{-j\omega t_a}\right| \tag{3.16}$$

$$|Y(\omega)| = |X(\omega)| \cdot \left|e^{-j\omega t_a}\right| \tag{3.17}$$

$$\left|e^{-j\omega t_a}\right| = \sqrt{\cos^2(-\omega t_a) + \sin^2(-\omega t_a)} = 1 \tag{3.18}$$

Final Conclusion

Substituting this unity magnitude back into the split equation gives:

$$|Y(\omega)| = |X(\omega)| \cdot 1 \tag{3.19}$$

$$|Y(\omega)| = |X(\omega)| \tag{3.20}$$

Looking at the given options, option (d) represents (3.16) before simplification:

$$\left|X(\omega) \cdot e^{-j\omega t_a}\right| = |X(\omega)| \cdot \left|e^{-j\omega t_a}\right| = |X(\omega)| \tag{3.21}$$

Therefore, the magnitude of the signal's frequency spectrum is entirely invariant to linear time delays, while phase shift is given by (3.1). The correct choice is **(d)**. This is illustrated in a plot using a Gaussian pulse as well as a sinc pulse as the input.

Python Implementation I

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq, fftshift, ifftshift
import os

output_dir = '../figs/'

dt = 0.01
t = np.arange(-50, 50, dt)

# sinc pulse
x = np.sinc(t)
t_a = 2.0
y = np.sinc(t - t_a) # y(t) = x(t - t_a)

n = len(t)
X = fftshift(fft(ifftshift(x)))
Y = fftshift(fft(ifftshift(y)))

freqs = fftshift(fftfreq(n, d=dt))

mag_X = np.abs(X)
```

Python Implementation II

```
mag_Y = np.abs(Y)

phase_X = np.angle(X)
phase_Y = np.angle(Y)

valid_idx = mag_X > 0.05*np.max(mag_X)

f_valid = freqs[valid_idx]
#ignore weak freqs instead of sharp jump to 0
phase_X_valid = np.unwrap(np.angle(X[valid_idx]))
phase_Y_valid = np.unwrap(np.angle(Y[valid_idx]))

center_idx = np.argmin(np.abs(f_valid)) # Set 0Hz to 0
phase_X_valid -= phase_X_valid[center_idx]
phase_Y_valid -= phase_Y_valid[center_idx]

plt.figure(figsize=(8, 5))
plt.plot(t, x, label='x(t) : Original Signal', color='blue')
plt.plot(t, y, label=f'y(t) : Delayed Signal (t_a={t_a})', color='red',
     linestyle='--')
plt.title('Time Domain')
plt.xlabel('Time (t)')
plt.ylabel('Amplitude')
```

Python Implementation III

```
plt.xlim(-5, 7)
plt.legend()
plt.grid(True)
plt.tight_layout()

time_domain_path = os.path.join(output_dir, 'time_domain_plot.png')
plt.savefig(time_domain_path, dpi=300)
print(f"Saved: {time_domain_path}")
plt.close()

plt.figure(figsize=(8, 5))
plt.plot(freqs, mag_X, label='|X()| : Magnitude of x(t)', color='blue',
     linewidth=3)
plt.plot(freqs, mag_Y, label='|Y()| : Magnitude of y(t)', color='orange',
     linestyle='--', linewidth=2)
plt.title('Frequency Domain: Magnitude')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.xlim(-2, 2)
plt.legend()
plt.grid(True)
plt.tight_layout()
```

Python Implementation IV

```
magnitude_path = os.path.join(output_dir, 'magnitude_spectrum_plot.png')
plt.savefig(magnitude_path, dpi=300)
print(f"Saved: {magnitude_path}")
plt.close()

plt.figure(figsize=(8, 5))
plt.plot(f_valid, phase_X_valid, label=r'$\angle X(\omega)$ : Phase of x(t)',
     color='blue') # waaaow latex crajee
plt.plot(f_valid, phase_Y_valid, label=r'$\angle Y(\omega)$ : Phase of y(t)',
     color='red', linestyle='--')
plt.title('Frequency Domain: Phase')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Phase (Radians)')
plt.xlim(-2, 2)
plt.legend()
plt.grid(True)
plt.tight_layout()

phase_path = os.path.join(output_dir, 'phase_spectrum_plot.png')
plt.savefig(phase_path, dpi=300)
print(f"Saved: {phase_path}")
plt.close()
```

Time Domain Plot

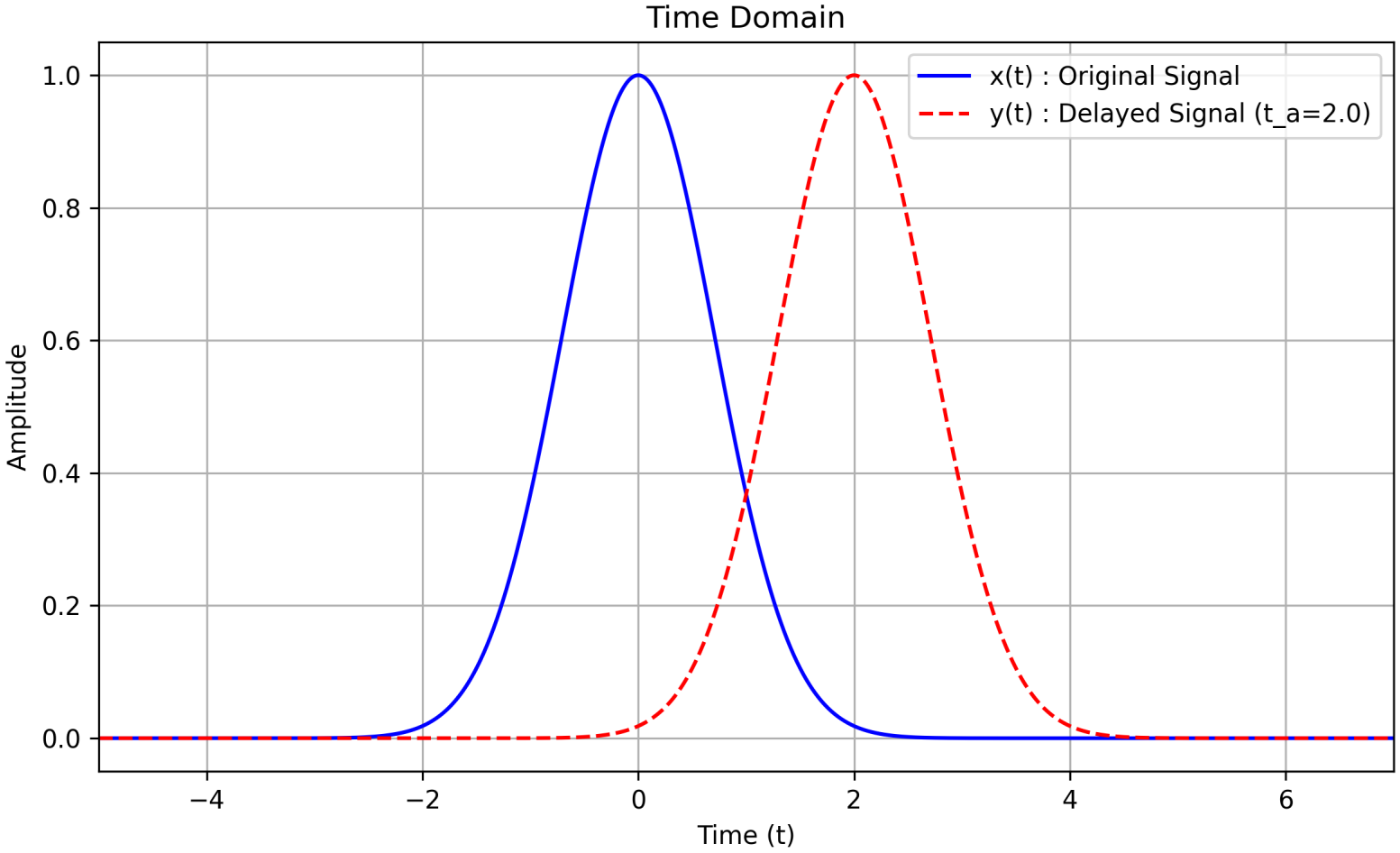

Figure: Time Domain: Original vs Delayed Signal

Magnitude Spectrum Plot

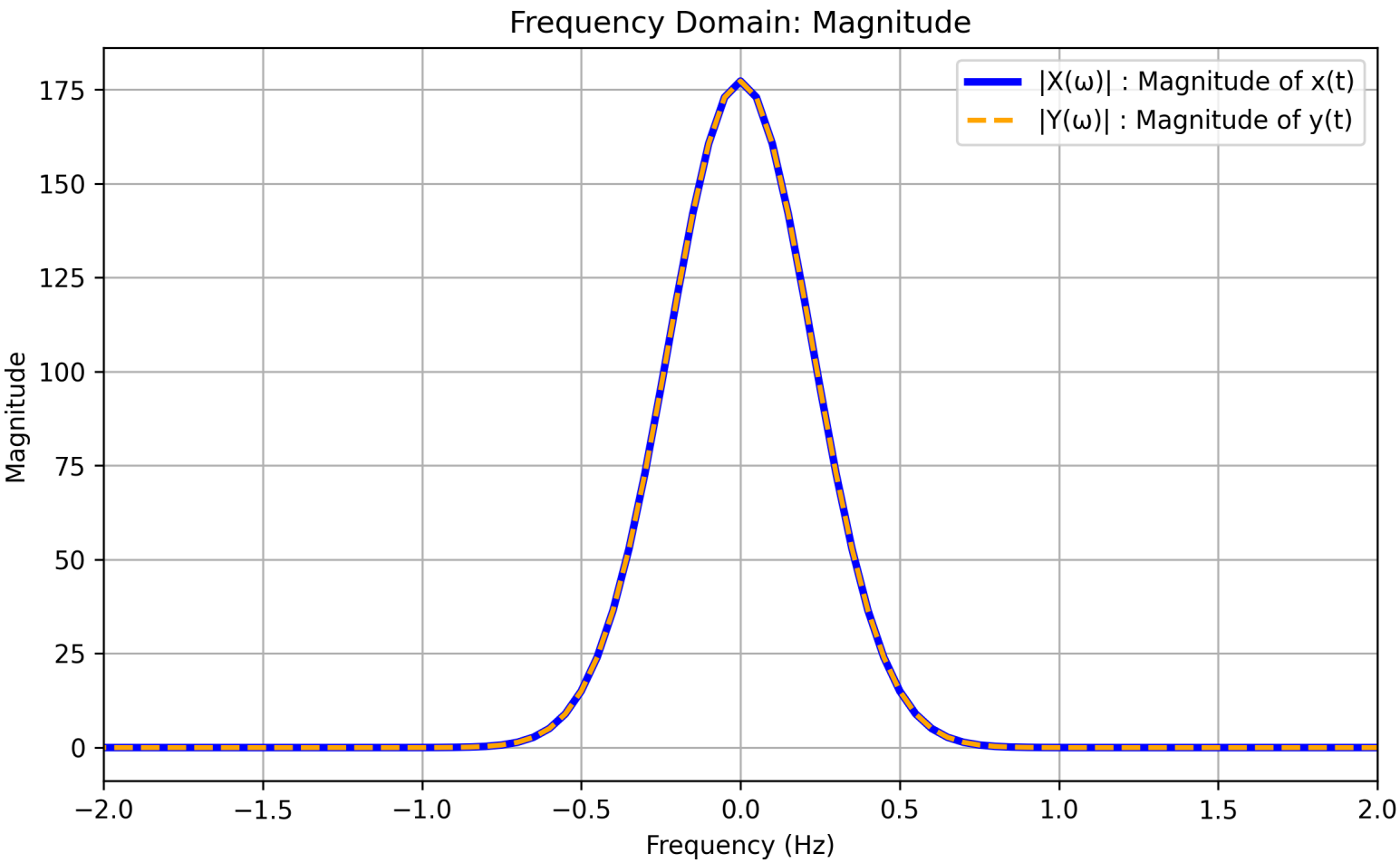

Figure: Frequency Domain: Magnitude Spectrum

Phase Spectrum Plot

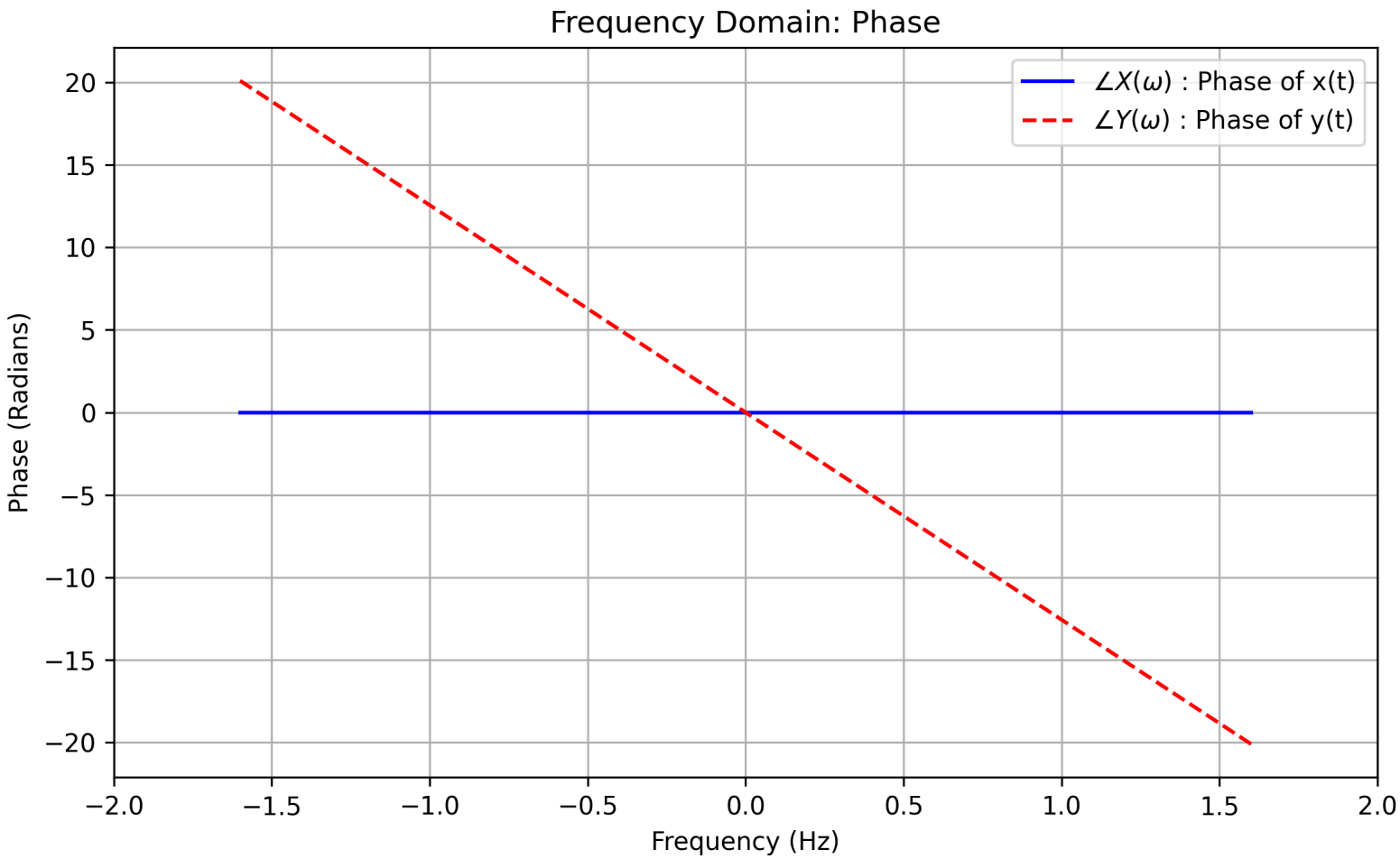

Figure: Frequency Domain: Phase Spectrum

Time Domain Plot

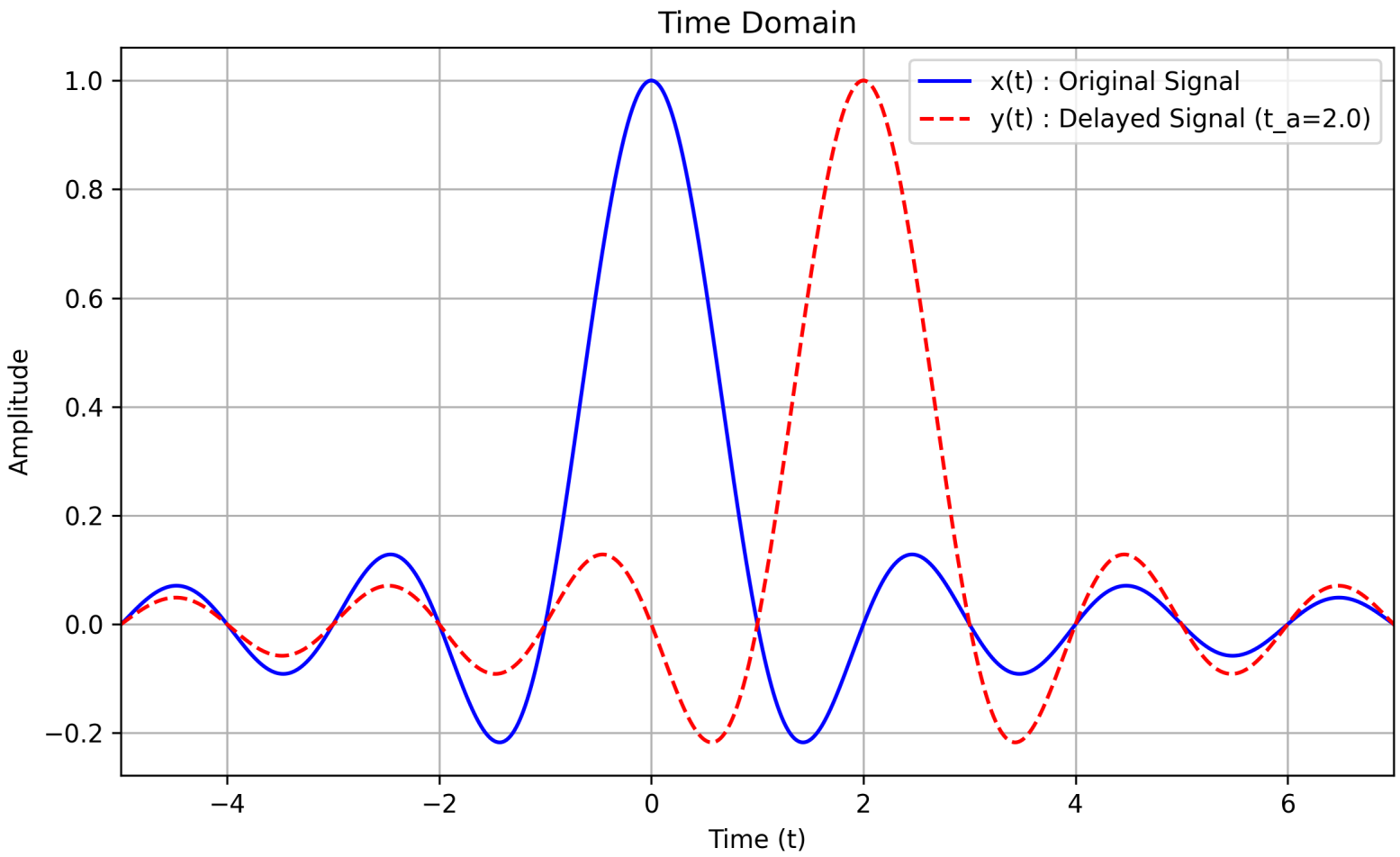

Figure: Time Domain: Original vs Delayed Signal

Magnitude Spectrum Plot

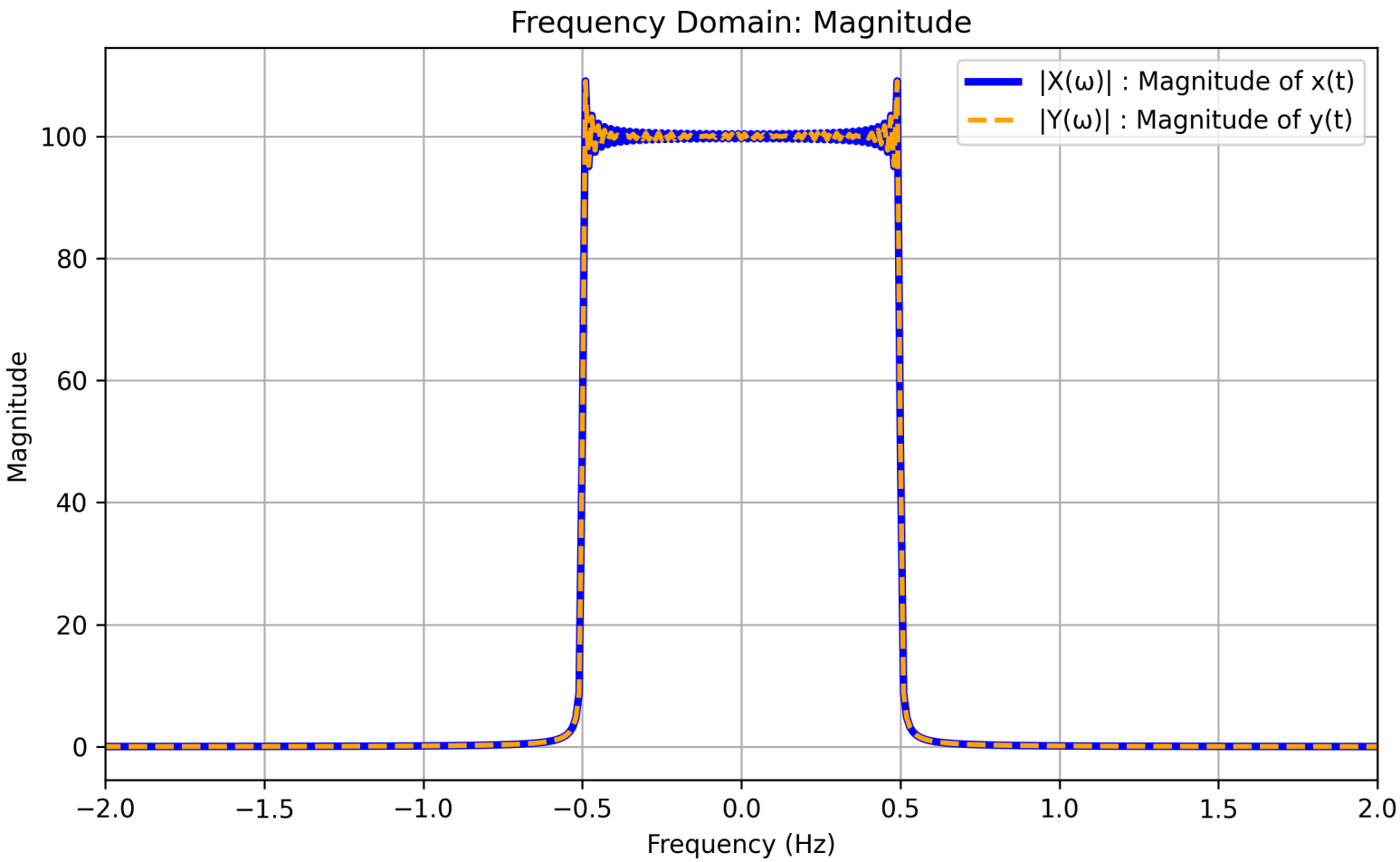

Figure: Frequency Domain: Magnitude Spectrum

Phase Spectrum Plot

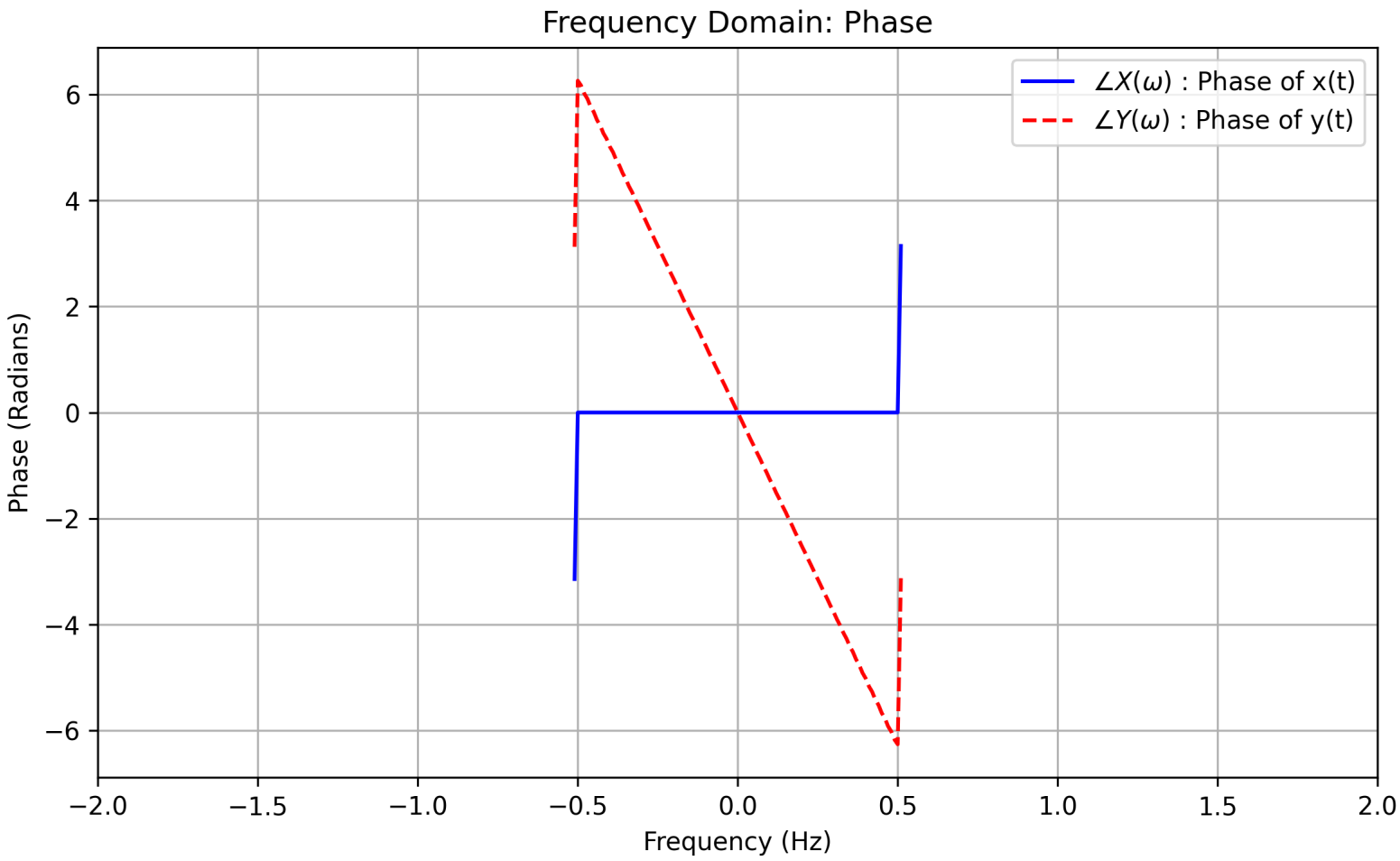

Figure: Frequency Domain: Phase Spectrum